



REPORT

형식언어 구현과제 1

제출일	2025 년 05 월 11 일	학과	컴퓨터공학전공
과목	형식언어	학번	2019111996
담당교수	손윤식 교수님	이름	김동훈

1. 주요 알고리즘

1. Tokenize 알고리즘

- 역할: 정규표현식 문자열을 토큰 리스트로 변환
- 주요 로직:
 - 연속된 심볼 사이에는 concatenation 연산자 삽입
 - is_symbol() 함수를 이용해 문자가 알파벳/숫자인지 확인
- 목적: 파싱 과정에서 이항/단항 연산자를 구분
- 코드:

```
def tokenize(re):
    tokens = []
    prev = ""
    for i, c in enumerate(re):
        # 접속 연산자 '.' 삽입 조건:
        if prev and (
            (is_symbol(prev) or prev == ')' or prev == '*') and
            (is_symbol(c) or c == '(')
        ):
            tokens.append('.')
        tokens.append(c)
        prev = c
    return tokens
```

2. 파서 및 구문 트리 생성 알고리즘

- 사용 알고리즘: shuntin yard(Dijkstra)
- 역할: 중위 표기법을 후위 표기법으로 변환하며 트리 구성
- 주요 로직:
 - 스택(ops)를 사용하여 연산자 우선순위를 관리
 - 단항 연산자(*)는 하나의 자식
 - 이항 연산자(+ , .) 는 좌우 자식
- 코드:

```
# 파서 + 트리 생성
def parse(tokens):
    output = []
    ops = []

    def pop_op():
        op = ops.pop()
        if op == '*':
            right = output.pop()
            output.append(Node(op, right))
        else:
            right = output.pop()
            left = output.pop()
            output.append(Node(op, left, right))

    for token in tokens:
        if is_symbol(token):
            output.append(Node(token))
        elif token == '(':
            ops.append(token)
```

```

elif token == ')':
    while ops and ops[-1] != '(':
        pop_op()
    ops.pop()
elif token in precedence:
    while (
        ops and ops[-1] in precedence and
        precedence[ops[-1]] >= precedence[token]
    ):
        pop_op()
    ops.append(token)

while ops:
    pop_op()

return output[0]

```

2. 실험 결과

- Arc(상태전이함수) 수의 상한 및 Size 계산:

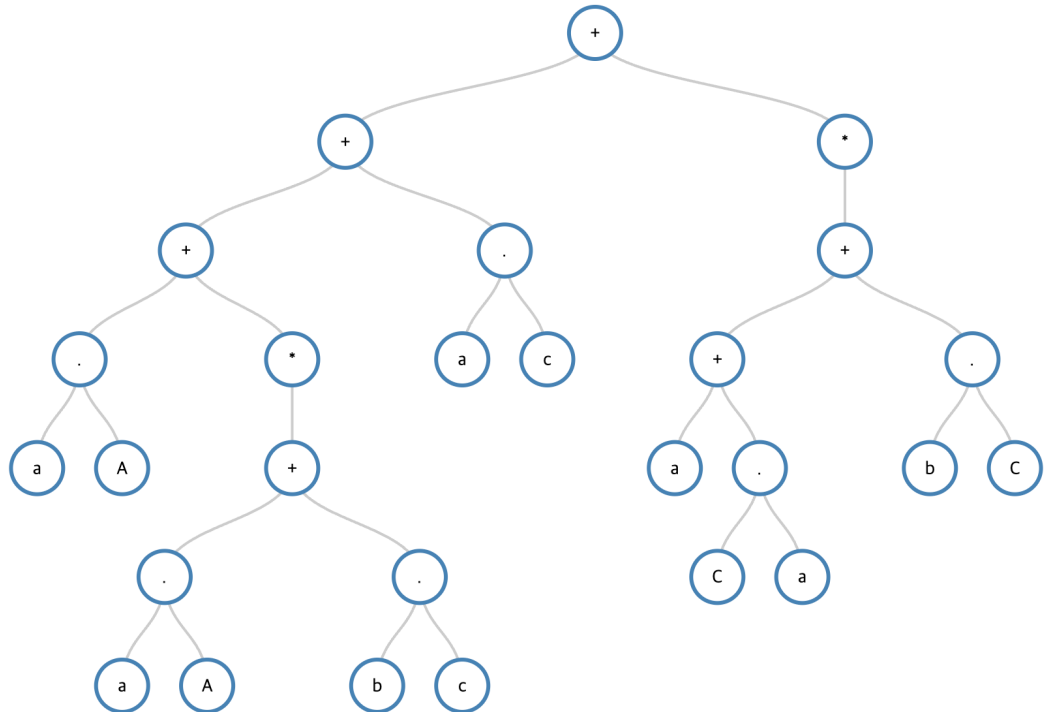
```

정규 표현 : aA+(aA+bc)*+ac+(a+Ca+bC)*
size : 27
arc 상한 : 108
실제 arc 수 : 42

```

- D3.js 를 이용한 자료구조 시각화:

정규 표현 트리 D3



정규 표현식: $aA+(aA+bc)^*+ac+(a+Ca+bC)^*$

3. 결과 분석

1. Arc 수의 상한 및 Size 계산

- 정규표현식: $aA+(aA+bc)^*+ac+(a+Ca+bC)^*$

항목	값	설명
size	총 노드 수 = 연산자 + 피연산자	구문 트리의 전체 노드 수
arc 상한	size × 4	각 노드가 최대 4 개의 arc 를 가질 수 있다는 가정 하의 이론적 최대
실제 arc 수	+는 4 개, *는 3 개, .는 2 개 기준으로 계산	각 연산자 타입별 고유 연결 수를 고려한 실제 값

- 위 과정은 주어진 정규 표현식이 상태 전이 함수(arc) 수 측면에서 얼마나 복잡한지 측정하기 위한 것이다. Arc 의 상한과 실제 arc 수를 비교함으로써, 이론적인 복잡도와 대비하여 실제 트리의 연결 구조가 얼마나 효율적인지 분석할 수 있다.
- 만약 $size = 27$, arc 상한 = 108, 실제 arc = 42 라면, 트리가 이론적인 상한인 108 개의 상태 전이 함수 수와 비교해 실제로는 42 개만 사용하여 효율적인 구조를 가졌음을 알 수 있다.

2. D3.js 를 이용한 자료구조 시각화

- 분석 포인트:
 - 연산자별 깊이와 위치 및 서브트리의 균형 여부
 - 트리의 비대칭성 여부
- 결과:
 - 올바르게 정규표현식의 구문구조가 트리로 파싱되었음
 - Tokenize 및 parse 알고리즘이 올바르게 구현되었음